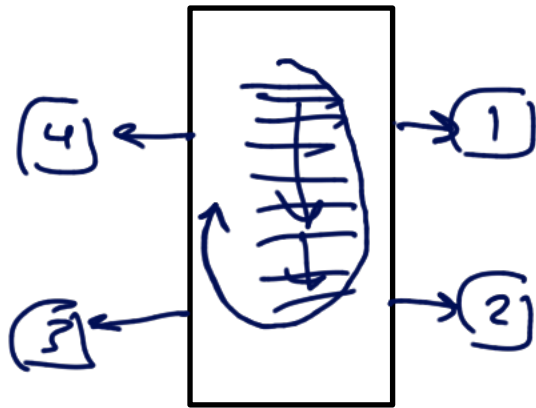


ch 11 interrupt

①

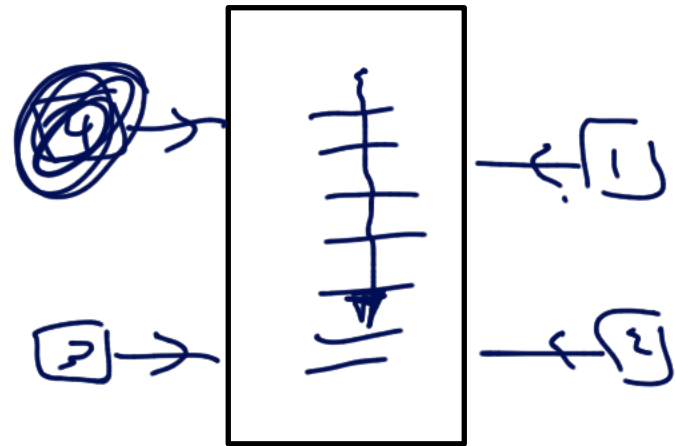
- 8051 can service many devices in two ways

① polling



- It wastes the 8051 (CPU) time
- There is no priority (round robin fashion)

② Interrupt



- It save the 8051 time
- There is a priority (programmable)

• ISR: Interrupt service routine (subroutine)

- 8051 has 6 types of interrupts (only 5 are programmable)
- for each interrupt, there must be an ISR.

- For every ISR there is a fixed locations in memory. (ROM).

(2)

- The group of memory locations that hold the addresses of ISRs is called Interrupt vector table.

Address	interrupt
0000H	Reset Interrupt
0003H	External hardware 0 interrupt (INT0) P3.2 (EX0)
000BH	Timer0 interrupt (TF0)
0013H	External hardware 1 interrupt (INT1) P3.3 (EX1)
001BH	Timer1 interrupt (TF1)
0023H	serial interrupt

programmable

by default $PC = 0000H$

address

Rom

(3)

0000H

JMP Main

} 3 byte

0003H

ISR
for
 $\overline{INT_0}$

} 8 byte

000BH

ISR
for
Timer 0

} 8

0013H

ISR
for
 $\overline{INT_1}$

} 8

001BH

ISR
for
Timer 1

} 8

0023H

ISR
for
serial
com

} 13 byte

002FH

Main : 0030H

main
program



FFFFH

ORG 0000H
LJMP Main

ORG 000BH
MOV R2, #15
MOV A, R2
MOV B, #10
RETI

} ISR

ORG 0030H
Main : MOV A, R3
INC A

MOV R7, #16H
MOV B, R7
SJMP \$

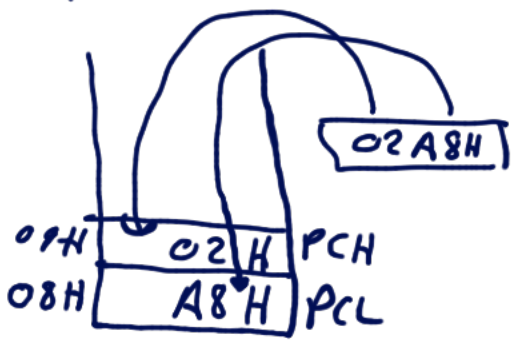


main program

R7 = 16H

① finish
R7 = 16H

② stack



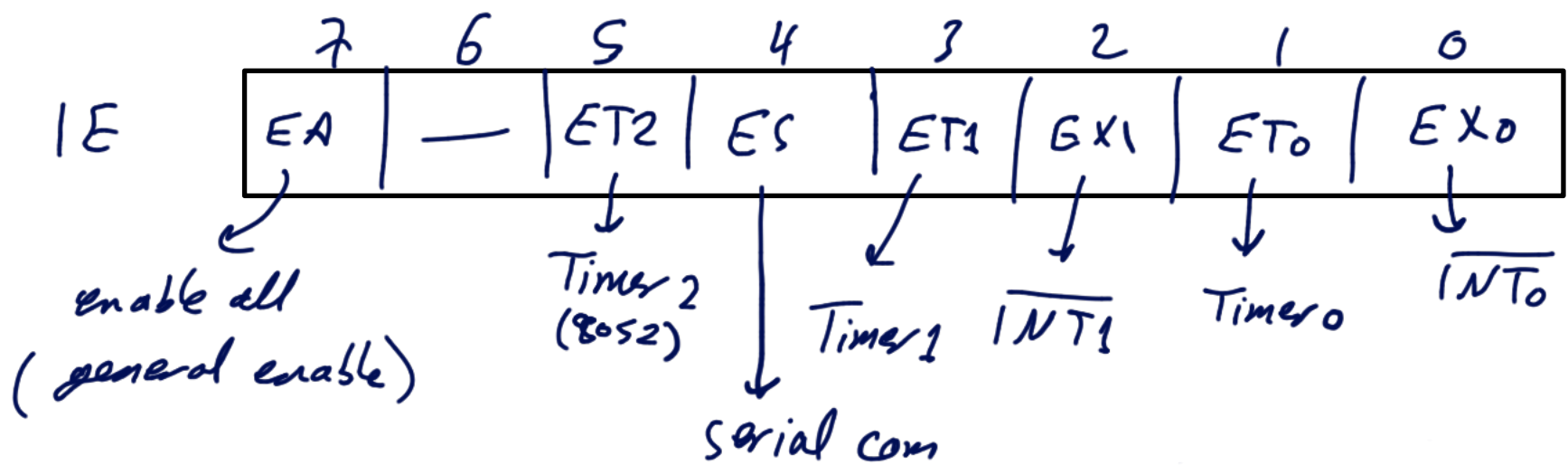
③ → Interrupt vector table
000BH

④ → execute ISR

⑤ → RETI
pop two bytes from stack into PC. To return the control to main program

⑤
- By default all interrupts are disabled.

- Programmer can enable/disable any interrupt using IE register (Interrupt Enable) → is bit addressable



Enable timer 0 interrupt and serial com interrupt

MOV IE, #10010010B

MOV IE, #92H

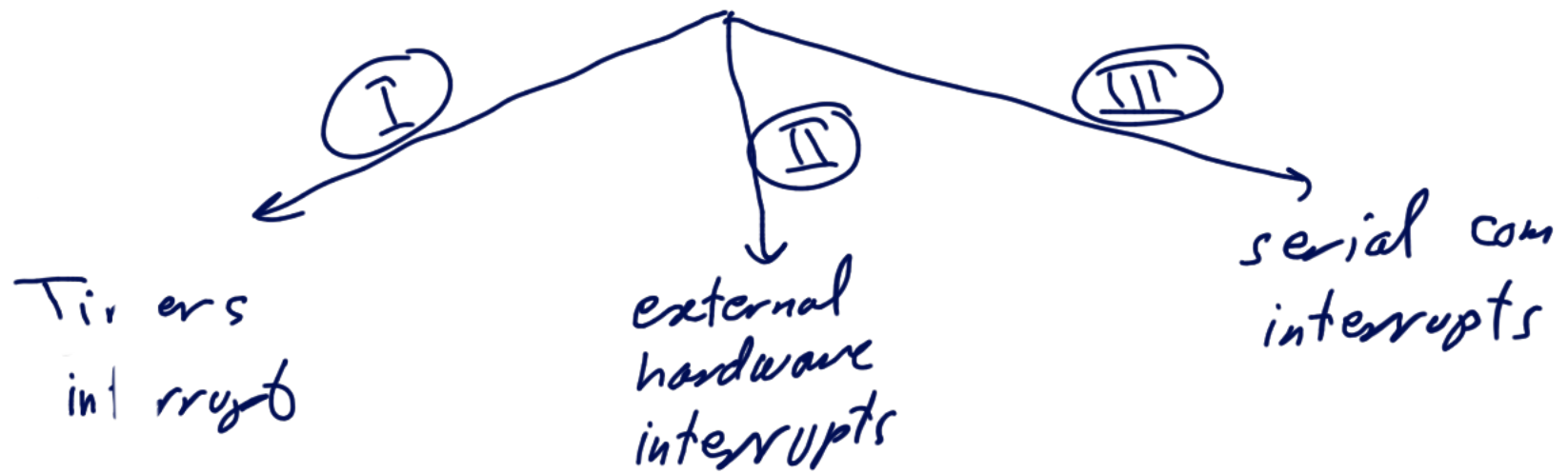
{ SETB IE.1
SETB IE.4
SETB IE.7

Disable serial com interrupt

→ MOV IE, #10000010B
→ CLR IE.4

Interrupt programs

⑥



I) Timer Interrupts:

* polling timer: 8086 will continuously monitor TF_i

HERE: $JNB \quad TF_i, \text{ HERE}$
 $\downarrow$
 001

* Interrupt:

TF_0

1

 $\rightarrow$ interrupt $\rightarrow PC = \underline{000BH}$

TF_1

1

 $\rightarrow$ " $\rightarrow PC = 001BH$

Write a program that continuously gets 8 bit data from P0 and send it to P1 while simultaneously creating a square wave of 200 μ s period on P2.1. Use Timer 0. Assume XTAL = 12 MHz.

$$MC = \frac{12}{12 \text{ MHz}} = 1 \mu\text{s}$$

$$\text{Delay} = \frac{T}{2} = \frac{200 \mu\text{s}}{2} = 100 \mu\text{s}$$

$$\frac{\text{Delay}}{MC} = \frac{100 \mu\text{s}}{1 \mu\text{s}} = 100 < 256 \rightarrow \text{mode 2 Timer 0}$$



② $\rightarrow$ MOV TH0, # -100

```
ORG 0000H
LJMP Main
```

```
ORG 000BH
CPL P2.1
RETI
```

```
ORG 0030H
Main : MOV P0, #0FFH ; input
      MOV TMOD, #0000 0010B
      MOV TH0, #-100
      MOV IE, #100000010B
      SETB TR0
Back : MOV A, P0
      MOV P1, A
      SJMP Back
```

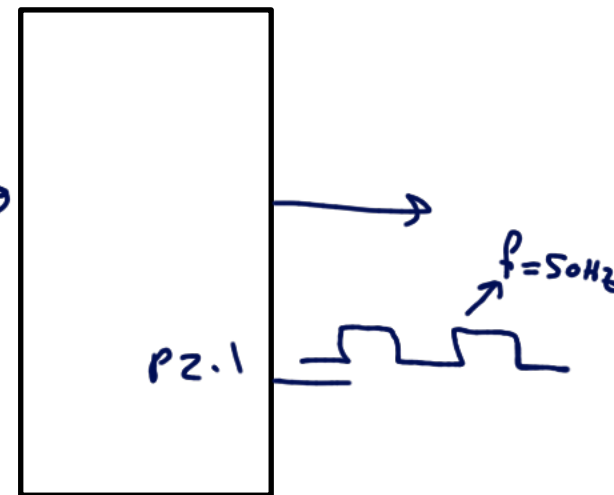
$TF_0 = 1$

⑨
- write a program to generate square wave of
50 Hz on P2.1 using interrupt. use timer 1.
Assume crystal frequency = 24 MHz.

$$Mc = \frac{12}{24 \text{ MHz}} = 0.5 \mu\text{s}$$

$$T = \frac{1}{f} = \frac{1}{50 \text{ Hz}} = 0.02 \text{ sec}$$

$$\text{Delay} = \frac{T}{2} = \frac{0.02 \text{ sec}}{2} = 0.01 \text{ sec}$$



$$\frac{\text{Delay}}{Mc} = \frac{0.01 \text{ sec}}{0.5 \mu\text{s}} = \underline{20000} > 256 \rightarrow \text{mode 1}$$

$$65536 - 20000 = 45536$$

$$\begin{array}{c} \curvearrowright = \text{BIE0H} \\ \downarrow \quad \downarrow \\ \text{TH1} \quad \text{TL1} \end{array}$$

ORG 0000H
LJMP Main

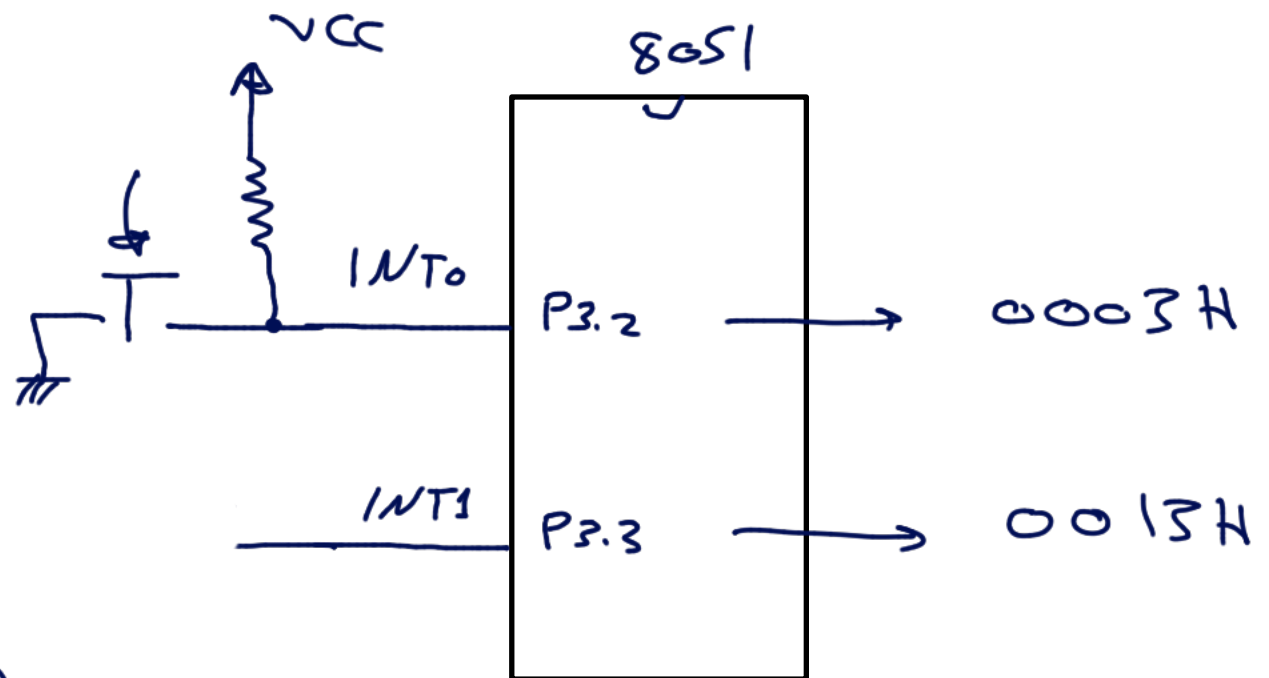
ORG 001BH
LJMP SQR

ORG 0030H
Main : MOV TMOD, #0001 0000B $\nearrow$ 10H
MOV TLI, #0E0H
MOV TH1, #0B1H
MOV IE, #10001000B $\nearrow$ 88H
SETB TRI
HERE: SJMP HERE

SQR: CLR TRI
CPL P2.1
MOV TLI, #0E0H
MOV TH1, #0B1H
SETB TRI
RETI

② External Interrupt

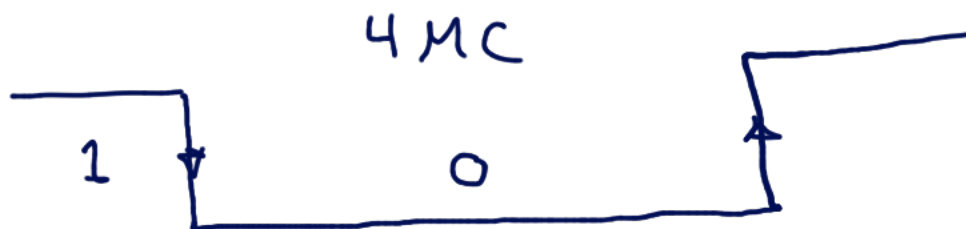
- 8051 has two external interrupts:



P3.2 must be 1 } no interrupt
P3.3 " " 1 }

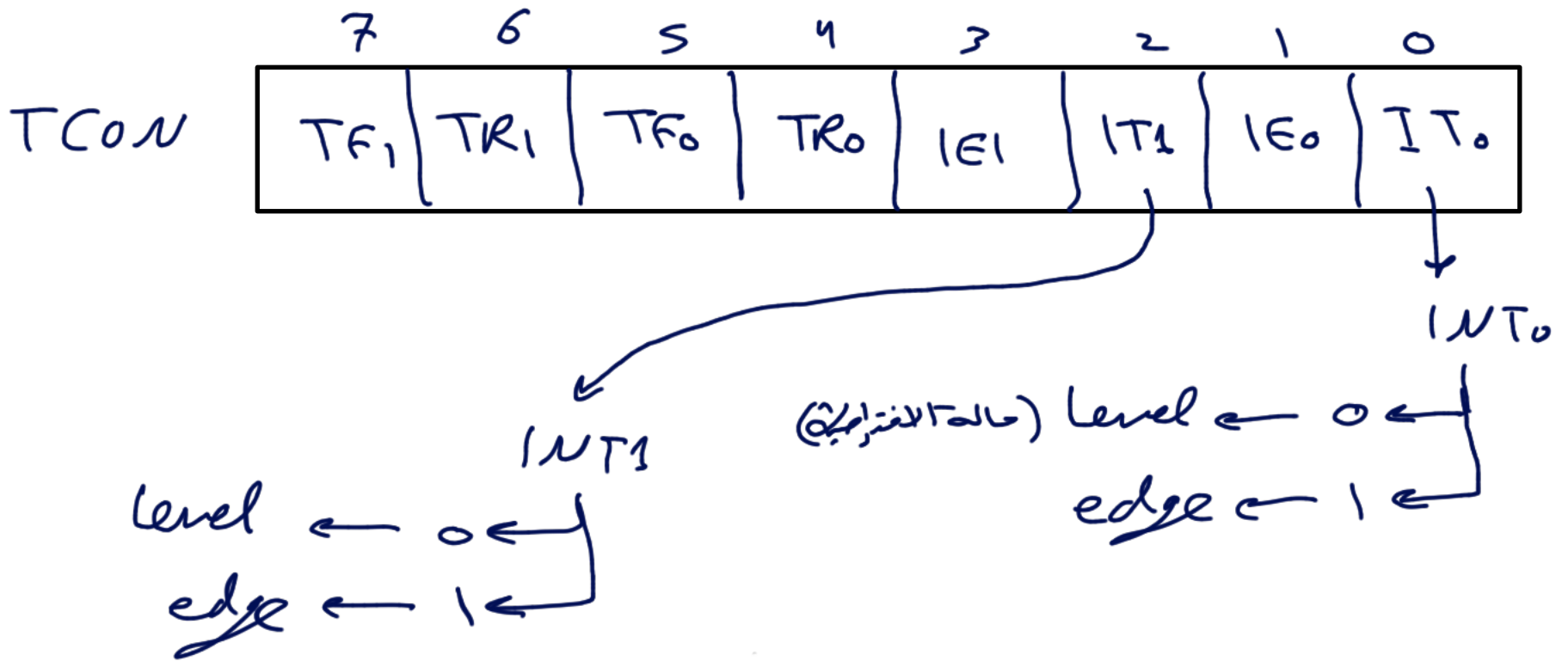
There are two ways for activating external interrupt

① Level triggered (by default)



② Edge triggered (falling edge)





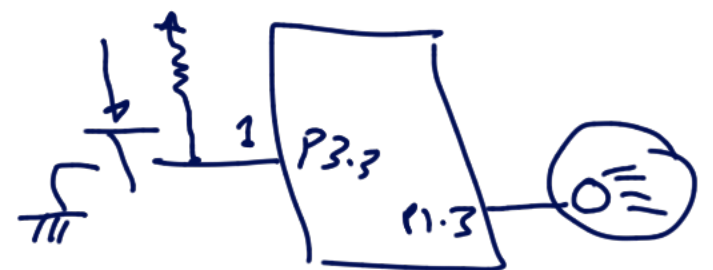
activate external interrupt 0 to be edge

SETB TCON.0

INT1 pin is connected to switch that is normally high. whenever it goes low (0) it should turn on LED on (P1.3). when it is turned on it should stay on for a fraction of seconds.

As long as the switch is pressed low the LED should stay on.

(level)



ORG 0000H
LJMP Main

ORG 0013H
LJMP LEDON

ORG 0030H
main : CLR P1.3 ; LED is off
SETB TCON.2 → MOV IE, # 10000100B
SJMP \$

LEDON : SETB P1.3 ←
MOV R2, #250 } delay
HERE : DJNZ R2, HERE
CLR P1.3 ←
RET



compare between RET and RETI ?

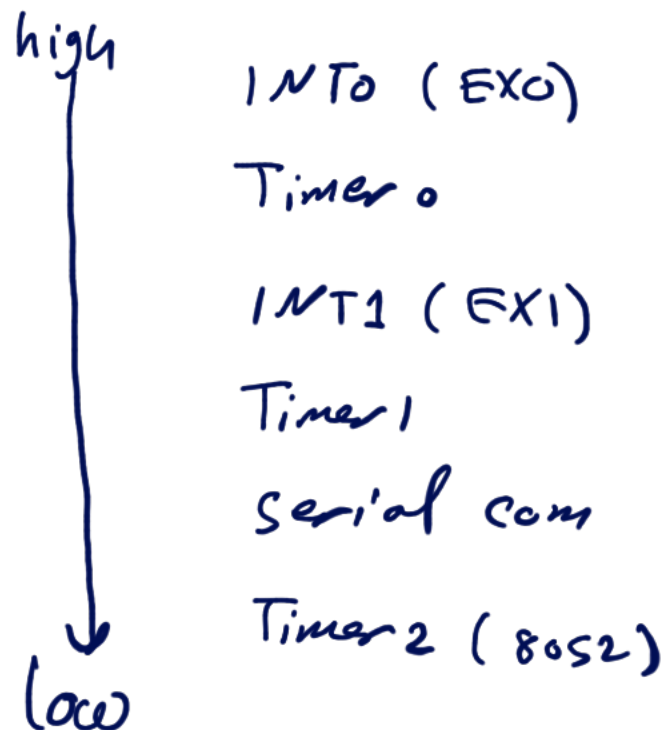
(14)

RET: return to main program only.

RETI: return to main program in addition to clear
TF0 and TF1 in timers interrupts and clear
TCOV.1 (ET0) and TCOV.3 (ETI) in edge
triggered external interrupts.

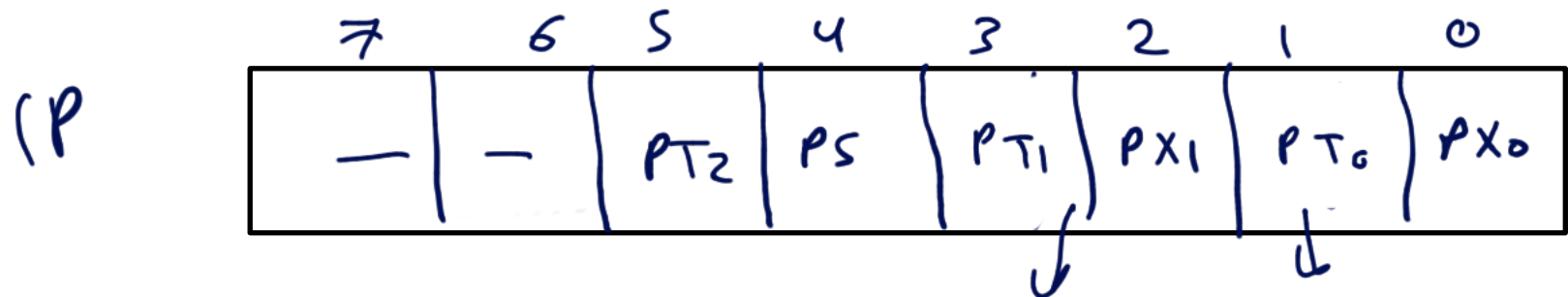
Interrupt priority

Default priority of interrupts



- programmer can change priority using IP

IP: Interrupt priority (8 bit)



by default IP = 00H

configure Timer 2 interrupt to be the highest priority

MOV IP, # 0000 1000B

→
Timer 1, INT0, Timer 0, INT1, Serial, Timer 2

MOV IP, # 0000 1010B

Timer 0, Timer 1, INT0, INT1, Serial, Timer 2

MOV IP, # 00111111B



by default priority
